Image credits: svitla.com

that do not require deployment in a production as it is the best option in such cases. It carries differences when compared to other databases. If the project is intended to go into production, then it is advisable to work on the same database that is to be used in production.

Apart from the databases already supported in Django, one can also use backends provided by third parties along with the database. Along with a database backend, we will require Python database bindings to be established. In the case of PostgreSQL, we need the Psycopg package. In MySQL, a database API such as mysqlclient is required. Oracle requires a copy of cx_Oracle, but don't forget to read its release notes on the recent updates of Oracle and cx_Oracle.

The requirement establishment of the system is done accordingly in case of any unofficial third-party backend. The manage.py migrate command creates a database automatically for the models after installation of Django and takes care of the database requirement. We need to permit Django to create and modify tables in the database that is being used. The database can also be created manually by feeding details in the setting file.

Let's consider a case of Postgresql database:

After installing the Postgresql database engine, one of the tools available is pgAdmin. The pgAdmin allows users to manage and modify databases locally. The primary task of this tool is to create a database; so, in the following steps, we will create a database because we don't deploy databases through Django manage.py migrate. Let's go through the following steps to configure the database: